Basics

Variables, types, operators

Output & Input

`print("Hi")`	show text
`print(a, b, sep="-", end="")`	print options
`input("Name: ")`	read text

Variables & Types

`x = 5`	int (whole number)
`y = 3.14`	float (decimal)
`s = "text"`	str (text)
`ok = True`	bool (True/False)
`n = None`	no value
`type(x)`	get the type
`int("5")`	text to int
`float("3.2")`	text to float
`str(5)`	anything to text

Operators

`+  -  *  /`	math
`//  %  **`	floor, modulo, power
`==  !=  <  >`	compare
`and  or  not`	logic
`in  /  not in`	membership
`is  /  is not`	identity
`+=  -=  *=  /=`	assign shortcuts

Comments

`# comment`	one line
`""" text """`	multi-line / docstring

Built-ins I

Numbers & conversion

Math

`abs(-5)`	absolute value
`round(3.14159, 2)`	round to 2 places
`pow(2, 3)`	power (8)
`divmod(7, 2)`	(quotient, rest)
`min(a, b)  max(a, b)`	smallest / largest
`sum([1, 2, 3])`	add everything

Convert

`bin(5)`	'0b101'
`hex(255)`	'0xff'
`oct(8)`	'0o10'
`int("ff", 16)`	parse any base
`chr(65)`	'A' from code
`ord("A")`	65 from char
`bool(x)`	truthiness
`complex(1, 2)`	complex number
`float("inf")`	infinity

Collections

`list(x)  tuple(x)`	to list / tuple
`set(x)  frozenset(x)`	unique items
`dict(a=1)`	make dict
`bytes(5)  bytearray(5)`	byte types

Built-ins II

Iterables & objects

Sequences

`len(x)`	length
`range(0, 10, 2)`	0,2,4,6,8
`sorted(x, reverse=True)`	new sorted list
`reversed(x)`	reverse iterator
`enumerate(x, start=1)`	index + item
`zip(a, b)`	pair up items

Functional

`map(f, x)`	apply f to each
`filter(f, x)`	keep where True
`any(x)`	any item True?
`all(x)`	every item True?
`iter(x)`	make iterator
`next(it, None)`	next item

Objects

`type(x)`	class of x
`isinstance(x, int)`	type check
`dir(x)`	list attributes
`getattr(o, "a", None)`	get attribute
`setattr(o, "a", 1)`	set attribute
`hasattr(o, "a")`	has attribute?

Run & Show

`repr(x)`	debug text
`format(3.5, ".2f")`	format value

Strings I

Case, search, edit

Case

`s.upper()  s.lower()`	change case
`s.title()`	Title Case
`s.capitalize()`	First letter up
`s.swapcase()`	flip case
`s.casefold()`	strong lowercase

Search

`s.find("a")`	index or -1
`s.rfind("a")`	search from right
`s.index("a")`	index or error
`s.count("a")`	how many times
`s.startswith("a")`	starts with?
`s.endswith("a")`	ends with?

Edit

`s.replace("a", "b")`	replace text
`s.strip()`	trim both sides
`s.lstrip()  s.rstrip()`	trim one side
`s.split(",")`	text to list
`s.splitlines()`	split on lines
`",".join(lst)`	list to text
`s.partition("=")`	split in 3 parts
`s.center(20, "*")`	pad centered
`s.ljust(10)  s.rjust(10)`	pad sides
`s.zfill(5)`	zero pad

Strings II

Checks & formatting

Check

`s.isdigit()`	only digits?
`s.isalpha()`	only letters?
`s.isalnum()`	letters + digits?
`s.isspace()`	only spaces?
`s.isupper()  s.islower()`	case check
`s.isnumeric()`	numeric chars?
`s.isidentifier()`	valid var name?

Format

`f"{x}"`	f-string
`f"{x:.2f}"`	2 decimals
`f"{x:>10}"`	right align
`f"{n:,}"`	1,000 style
`f"{x!r}"`	repr inside
`"{} {}".format(a, b)`	format()
`"%s" % x`	old style

Slice & Encode

`s[1:3]`	chars 1 to 2
`s[::-1]`	reverse text
`s[-1]`	last char
`s * 3`	repeat
`s.encode()`	to bytes
`b.decode()`	bytes to text
`s.translate(table)`	map chars
`str.maketrans("a", "b")`	build table
`"a" in s`	contains?

Lists

Ordered & changeable

Add / Remove

`lst.append(x)`	add to end
`lst.extend(it)`	add many
`lst.insert(i, x)`	insert at i
`lst.remove(x)`	remove first x
`lst.pop()`	remove & return
`del lst[0]`	delete by index
`lst.clear()`	empty it

Find / Order

`lst.index(x)`	position of x
`lst.count(x)`	times x appears
`lst.sort(reverse=True)`	sort in place
`lst.reverse()`	reverse in place
`lst.copy()`	shallow copy
`sorted(lst, key=len)`	new sorted list

Slice & Combine

`lst[1:3]`	items 1 and 2
`lst[::-1]`	reversed copy
`lst[-1]`	last item
`a + b`	join lists
`lst * 2`	repeat
`[0] * 5`	five zeros
`a, b = lst`	unpack
`first, *rest = lst`	star unpack
`x in lst`	contains?
`[[0]*3 for _ in range(3)]`	2D grid

Tuples & Sets

Fixed and unique

Tuples

`t = (1, 2, 3)`	immutable list
`t[0]`	get item
`t.count(x)`	times x appears
`t.index(x)`	position of x
`a, b = b, a`	swap values

Sets

`s = {1, 2, 3}`	unique items
`s.add(x)`	add item
`s.update(it)`	add many
`s.remove(x)`	remove or error
`s.discard(x)`	remove safely
`s.pop()`	remove random
`s.clear()`	empty it

Set Math

`a | b`	union
`a & b`	intersection
`a - b`	difference
`a ^ b`	in one, not both
`a <= b`	subset?
`a.isdisjoint(b)`	nothing shared?
`list(set(lst))`	remove duplicates

Dictionaries

Key and value

Basics

`d = {"k": 1}`	key: value
`d["a"]`	get value
`d.get("k", 0)`	safe get
`d["k"] = 2`	set value
`"k" in d`	key exists?
`del d["k"]`	delete key

Methods

`d.keys()`	all keys
`d.values()`	all values
`d.items()`	key-value pairs
`d.pop("k")`	remove & return
`d.popitem()`	remove last
`d.update(d2)`	merge in place
`d.setdefault("k", 0)`	get or set
`d.copy()`	shallow copy
`d.clear()`	empty it
`dict.fromkeys(keys, 0)`	keys with a value

Patterns

`for k, v in d.items():`	loop pairs
`{**d1, **d2}`	merge dicts
`d1 | d2`	merge (3.9+)
`{k: v for k, v in x}`	dict comp
`sorted(d, key=d.get)`	keys by value
`max(d, key=d.get)`	key of max

Control Flow

Conditions & loops

Conditions

`if x > 0:`	check
`elif x == 0:`	else if
`else:`	otherwise
`a if cond else b`	one-line if
`match x:`	switch (3.10+)
`case 1:`	match a value

Loops

`for i in range(5):`	repeat 5 times
`for x in lst:`	loop items
`for i, x in enumerate(lst):`	index + item
`for a, b in zip(x, y):`	loop pairs
`while cond:`	loop while True
`break`	exit loop
`continue`	next round
`pass`	do nothing
`else: (after loop)`	ran without break

Comprehensions

`[x*2 for x in lst]`	list comp
`[x for x in lst if x > 0]`	with filter
`{x for x in lst}`	set comp
`{k: v for k, v in p}`	dict comp
`(x**2 for x in lst)`	generator
`if (n := len(lst)) > 3:`	assign inside

Functions

Define, call, decorate

Define & Call

`def f(a, b=1):`	default value
`return a + b`	send back
`def f(*args):`	any positional
`def f(**kw):`	any keyword
`def f(a, /, b, *, c):`	arg rules
`def f() -> int:`	type hint
`lambda x: x * 2`	tiny function
`f(a=1, b=2)`	keyword arg
`f(*lst)`	unpack list
`f(**d)`	unpack dict

Scope & Generators

`global x`	use global var
`nonlocal x`	use outer var
`yield x`	make generator
`yield from it`	delegate
`async def f():`	coroutine
`await task`	wait for result

Decorators

`@decorator`	wrap a function
`@staticmethod`	no self
`@classmethod`	gets cls
`@property`	getter
`@wraps(fn)`	keep metadata
`@lru_cache`	cache results
`"""doc"""`	docstring

Classes (OOP)

Objects & magic methods

Classes

`class A:`	define class
`def __init__(self, x):`	constructor
`self.x = x`	instance attribute
`def method(self):`	method
`class B(A):`	inherit
`super().__init__(x)`	call parent

Magic Methods

`def __str__(self):`	print text
`def __repr__(self):`	debug text
`def __len__(self):`	len(obj)
`def __eq__(self, o):`	==
`def __lt__(self, o):`	<
`def __add__(self, o):`	+
`def __getitem__(self, i):`	obj[i]
`def __iter__(self):`	for loop
`def __call__(self):`	obj()
`def __enter__(self):`	with start
`def __exit__(self, *a):`	with end

Extras

`@dataclass`	auto class
`isinstance(o, A)`	type check
`type(o).__name__`	class name
`__slots__ = ("a",)`	save memory
`A.x = 1`	class attribute

Errors & Files

Try, except, open

Exceptions

`try:`	risky code
`except ValueError as e:`	catch error
`except (A, B):`	catch many
`else:`	no error happened
`finally:`	always runs
`raise ValueError("x")`	throw error
`assert x > 0, "msg"`	debug check
`class MyErr(Exception):`	custom error

Common Errors

`ValueError  TypeError`	bad value / type
`KeyError  IndexError`	missing key / index
`ZeroDivisionError`	divide by zero
`FileNotFoundError`	file missing
`ImportError`	module missing

Files

`with open(p) as f:`	auto close
`open(p, "r")`	read (w a b +)
`f.read()`	whole file
`f.readline()`	one line
`f.readlines()`	list of lines
`f.write("text")`	write text
`for line in f:`	loop lines
`f.seek(0)`	move cursor
`Path(p).read_text()`	pathlib read

Modules I

math, random, datetime

math

`import math`	load module
`math.sqrt(x)`	square root
`math.pi  math.e`	constants
`math.ceil(x)  math.floor(x)`	round up / down
`math.factorial(n)`	n!
`math.gcd(a, b)`	greatest divisor
`math.log(x, b)`	logarithm
`math.sin(x)`	trig (cos, tan)

random

`random.random()`	0 to 1
`random.randint(1, 10)`	int in range
`random.choice(lst)`	pick one
`random.sample(lst, k)`	pick k unique
`random.shuffle(lst)`	shuffle in place
`random.uniform(1, 5)`	float in range
`random.seed(1)`	repeatable

datetime & time

`datetime.now()`	current time
`datetime(2026, 1, 1)`	make a date
`d.strftime("%Y-%m-%d")`	date to text
`datetime.strptime(s, f)`	text to date
`timedelta(days=7)`	time gap
`time.time()`	timestamp
`time.sleep(2)`	pause 2 sec

Modules II

os, sys, json, re, collections

os

`os.getcwd()`	current folder
`os.listdir(".")`	list folder
`os.makedirs("d")`	make folders
`os.remove("f")`	delete file
`os.path.join(a, b)`	build a path

sys & json

`sys.argv`	cli arguments
`sys.exit()`	quit
`json.dumps(d)`	dict to text
`json.loads(s)`	text to dict
`json.load(f)`	read json file

re (regex)

`re.search(p, s)`	first match
`re.findall(p, s)`	all matches
`re.sub(p, r, s)`	replace
`re.split(p, s)`	split by pattern

collections & itertools

`Counter(lst)`	count items
`defaultdict(list)`	auto default
`deque([1, 2])`	fast queue
`chain(a, b)`	join iterables
`permutations(x)`	all orders

Tips & Terminal

Tips & the terminal

Terminal

`python --version`	check version
`python file.py`	run a script
`pip install pkg`	install package
`pip list`	installed packages
`python -m venv env`	virtual env
`python -m http.server`	quick web server

Good Habits

`if __name__ == "__main__":`	script entry point
`import this`	zen of Python

Keep coding. Keep learning.

Made by

@hafizibrahimsuriya

Save this sheet | Share it | Practice daily